Technische Universität Berlin

Distributed Systems
Chapter 2: Cooperation and Coordination

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

2.2 Cooperation

- Message Passing
- Message Queues

Cooperation

The essence of a distributed system:

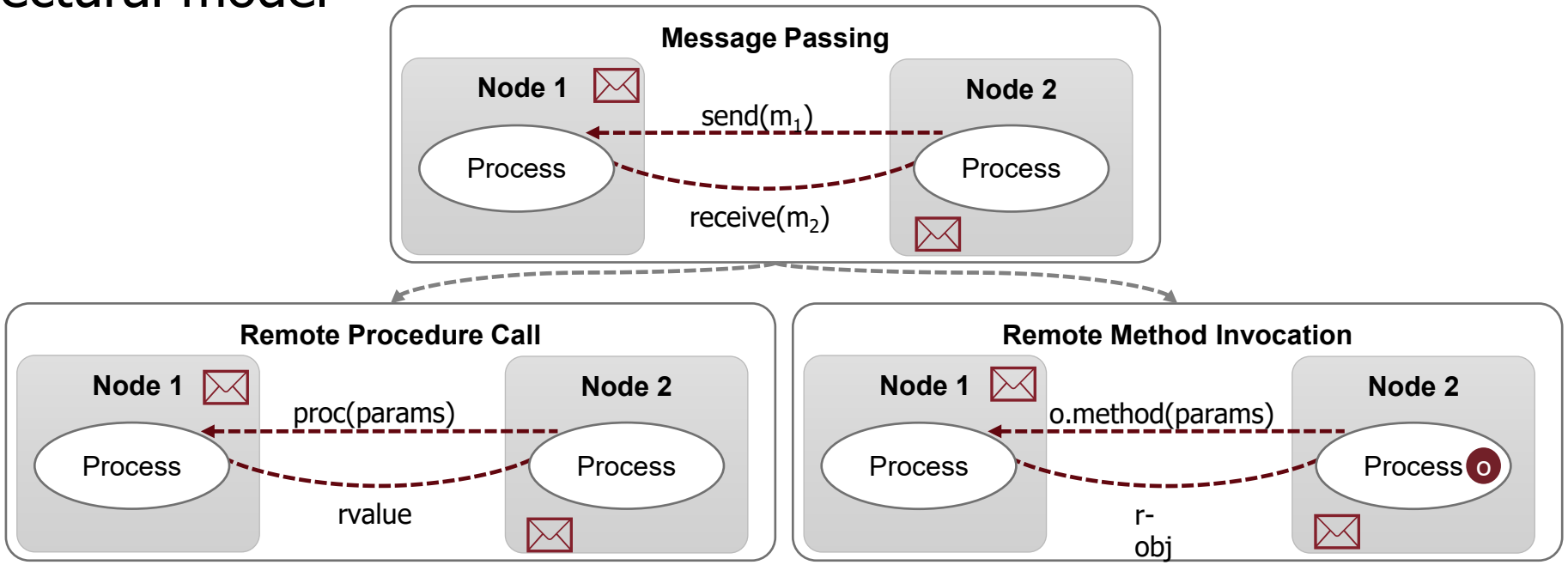
- Computation takes places in local processes
- Local processes interact and share results and resources by passing messages
- results in communication (information flow) and coordination (synchronization of activities)

Motivation for cooperation / communication implicit – remember definition:

„DS is one in which HW or SW components are located at networked computers and communicate and coordinate their actions only by passing messages."

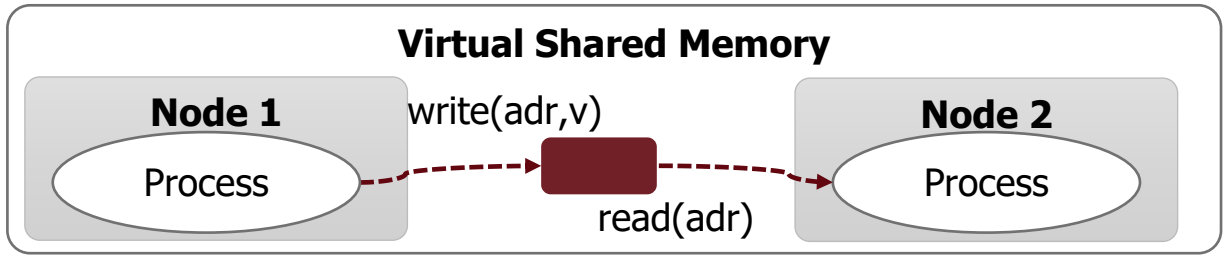
Models of Cooperation

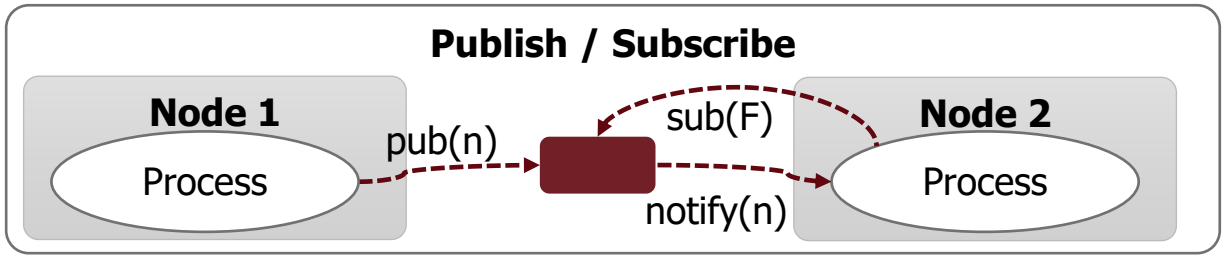
Basic models of cooperation defined as communication paradigms used by the architectural model



Models of Cooperation

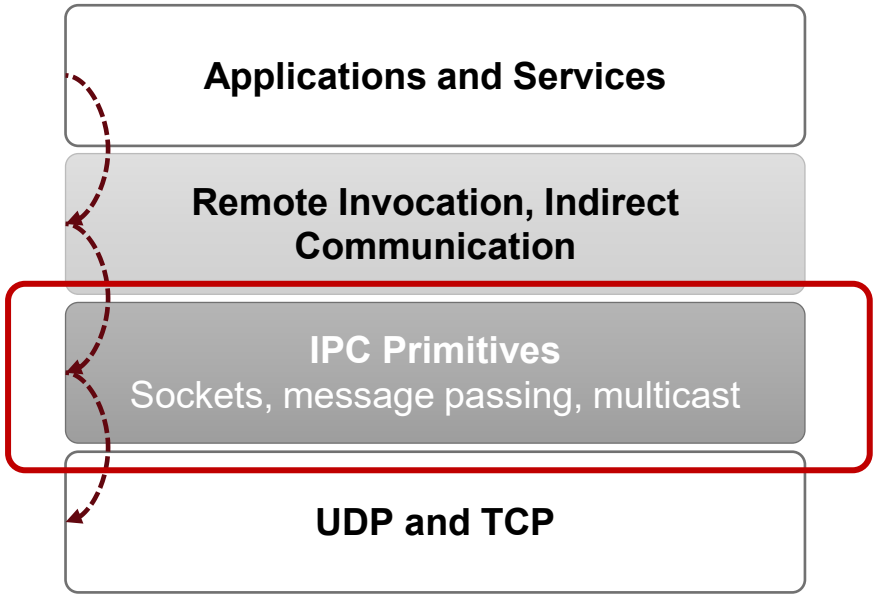
Further (incomplete) classification from the programmer's point of view





Message Passing

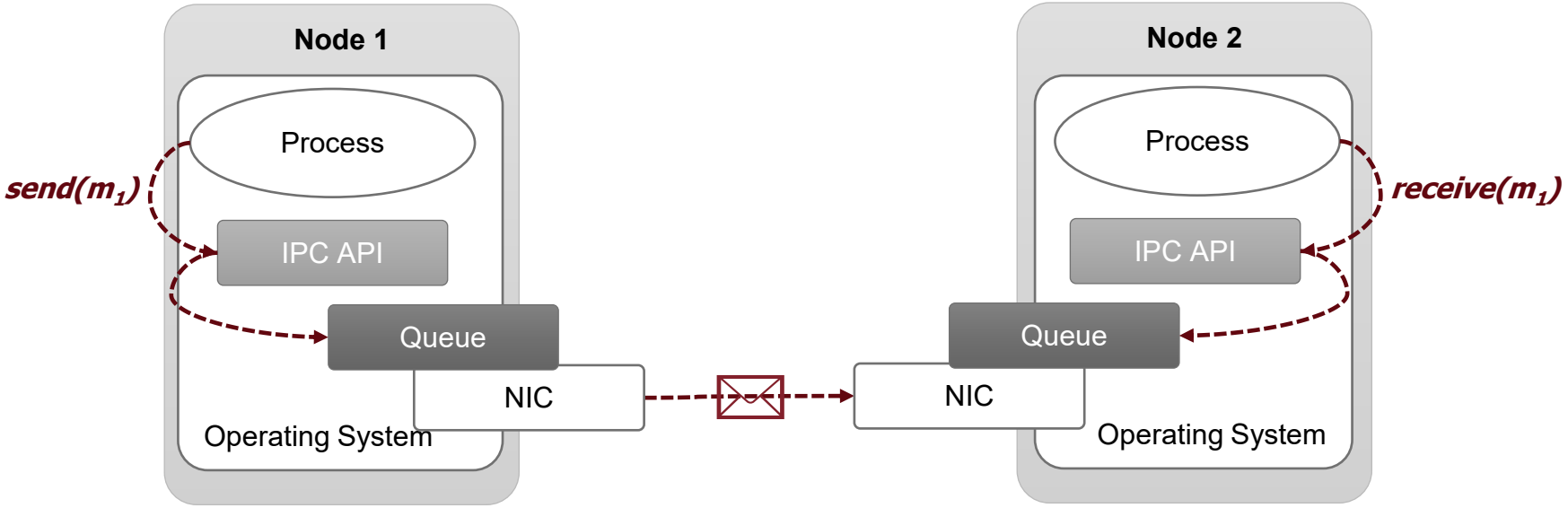
Cooperation (communication) in its simplest form is based on inter process communication (IPC) paradigms



Message Passing

Communication of data between a sending and a receiving process

– send(msg) and receive(msg) operations are provided by IPC API of the OS



IPC (Inter-Process Communication) API provides access to NIC

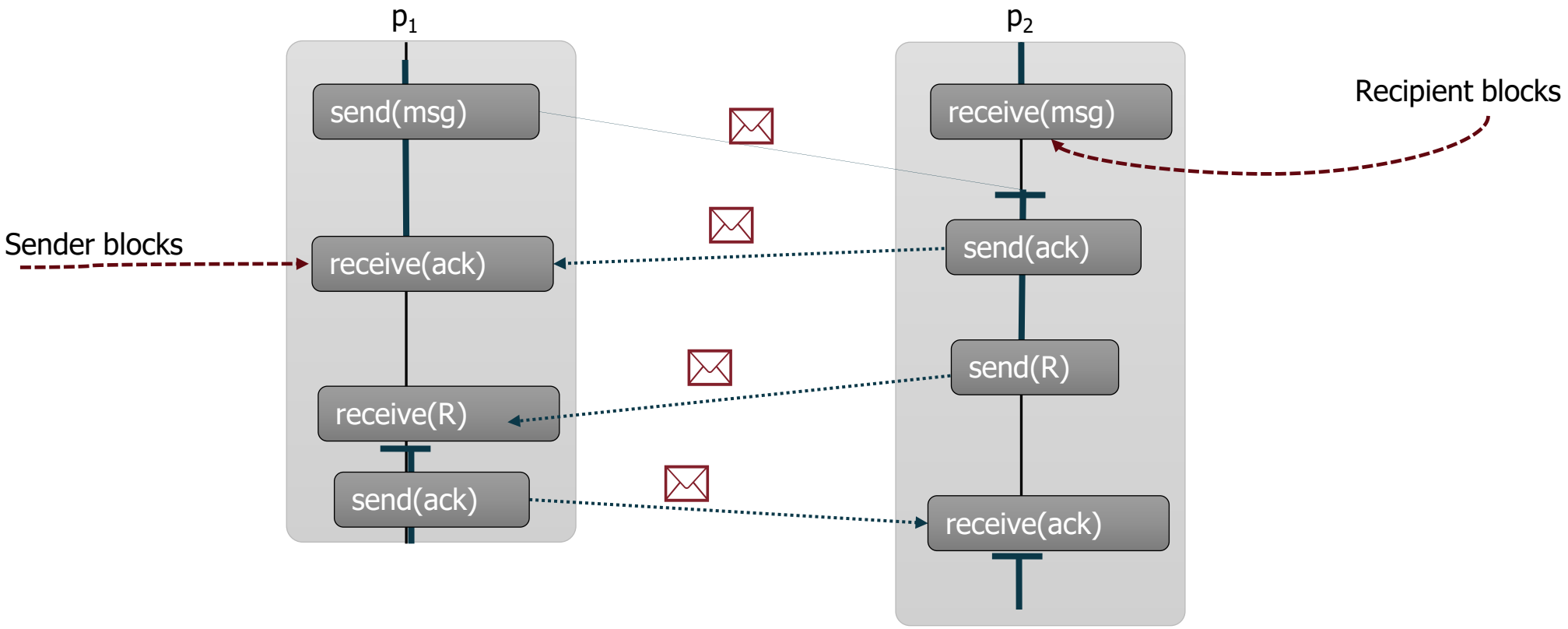
Message Passing: Synchronous Communication

send(msg) and receive(msg) are blocking operations

- A sending process is blocked until the corresponding receive is called
- A call to receive blocks until a message arrives

Can be transparently implemented by the IPC API using acknowledgements

Message Passing: Synchronous Communication



Message Passing: Synchronous Communication

Pros

- Sender knows that recipient has received message
- Only one message needs to be buffered
- Balances different processing performance through waiting

Cons

- Tight coupling: sender and recipient must run simultaneously
- Sender is blocked → low parallelism or complex multithreading, risk of deadlocks

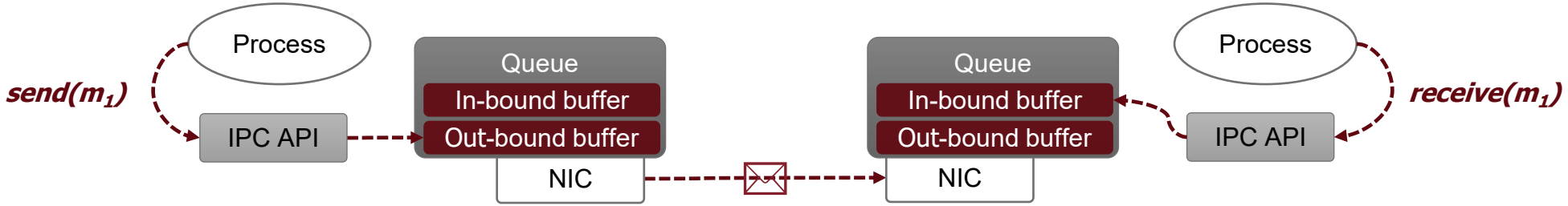
Message Passing: Asynchronous Communication

send(msg) is non-blocking

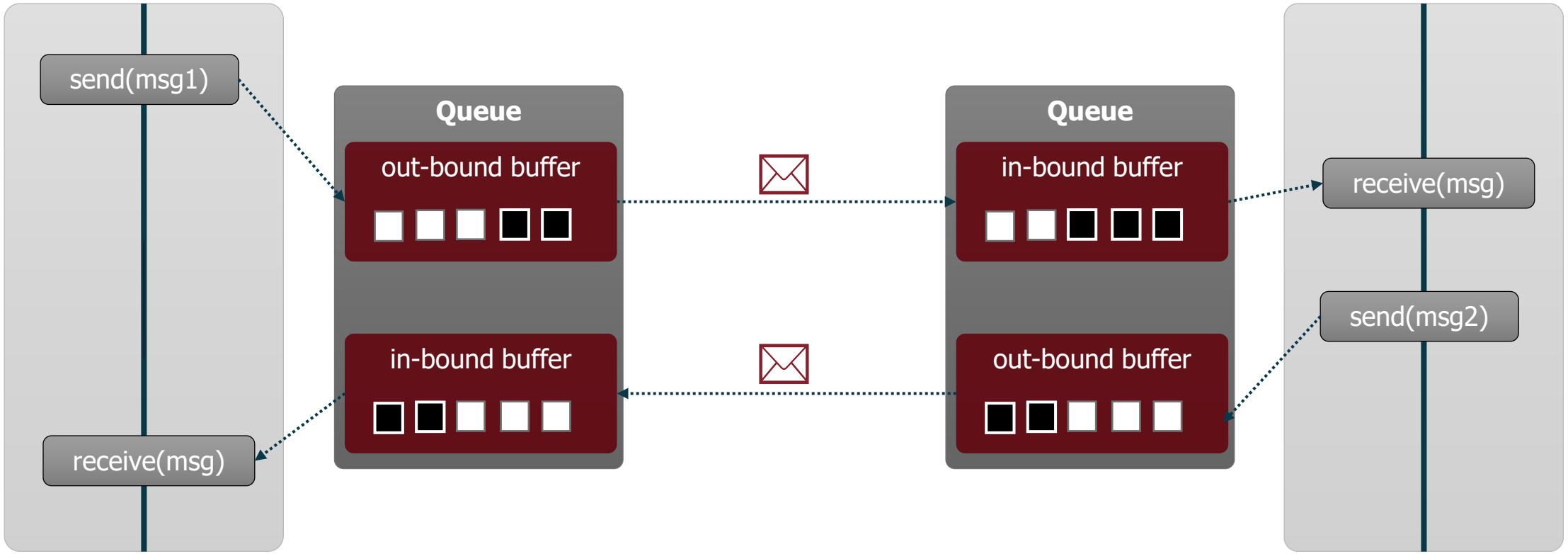
- send(msg) copies the message to a local buffer
- The transmission of the message proceeds in parallel with the sending process

receive(msg) can be blocking or non-blocking

- If non-blocking, receive(msg) provides a buffer to the IPC API, which callbacks/notifies the process once the buffer has been filled



Message Passing: Asynchronous Communication



Message Passing: Asynchronous Communication

Pros

- Loose coupling → no need for simultaneous operation of sender and receiver
- High degree of parallelism, reduced risk of deadlocks
- Out-bound buffer allows **temporarily faster** sending than actual transmission
- In-bound buffer allow temporarily faster receiving

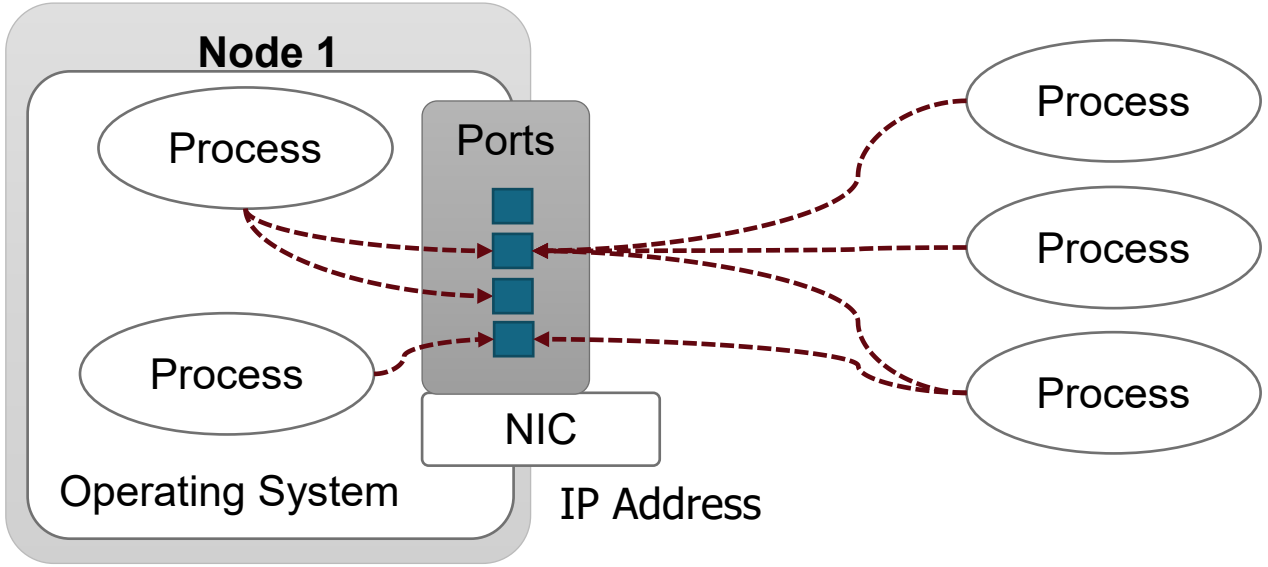
Cons

- Sender does not know if or when a message was received
- Buffer overflow possible
- Buffers balance current speed variance, but not permanent performance differences

Message Passing

Communication requires a message destination

Internet Protocols use <Internet Address, Local Port> as message destination



Message Passing

Some applications require messages are delivered in sender order

⇒Messages delivered out of sender order are considered faulty

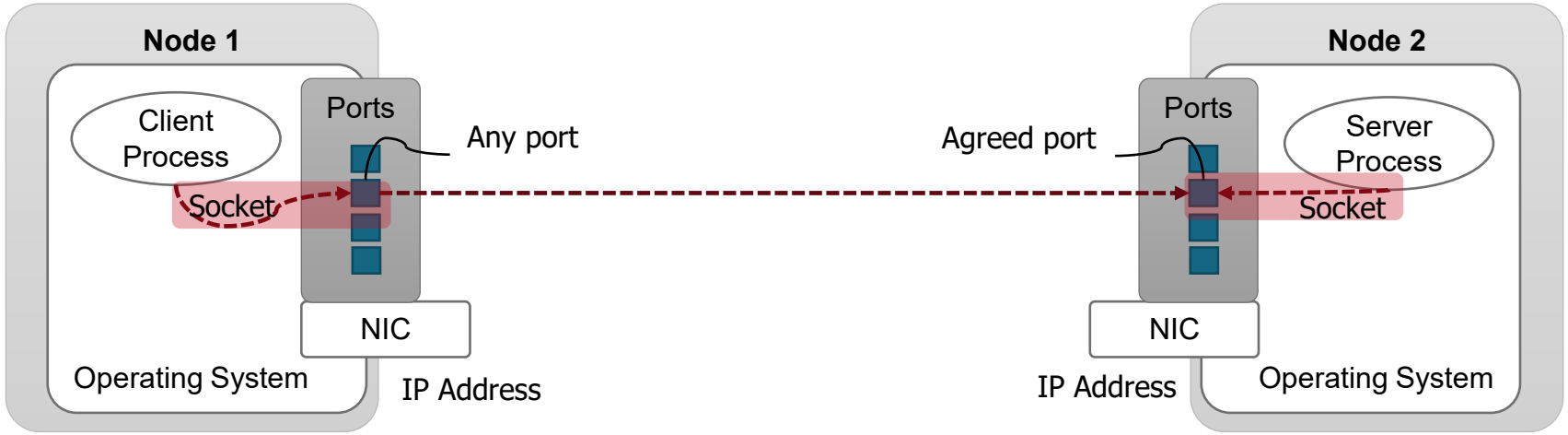
Reliability of a communication channel is defined as (according to the reliable communication definition of the failure model)

- Integrity: Messages must arrive uncorrupted and without duplication
- Validity:
  - A message passing service is reliable if messages are guaranteed to be delivered even if packets are dropped or lost (note that packet ≠ message)
  - A message passing service is unreliable if messages are not guaranteed to be delivered even if only a single packet is dropped or lost

Message Passing – Internet Protocols

On top of IP, TCP and UDP provide means for exchanging data between two nodes

- Both are based on the socket programming model
- Messages are exchanged between two sockets, each corresponding to a process



Message Passing – UDP

UDP as connection-less transport or datagram-oriented communication

Failure Model for UDP

- UDP offers a best-effort / unreliable transport service
- It suffers from send-omission and receive-omission failures as well as delivering out of sender order
- In terms of the validity and integrity properties defined by our failure model
  - Integrity: UDP uses checksums to detect corrupted messages (duplicates may occur due to issues with the network infrastructure – e.g. loops)
  - Validity: dropped or lost packets are not retransmitted

Message Passing – TCP

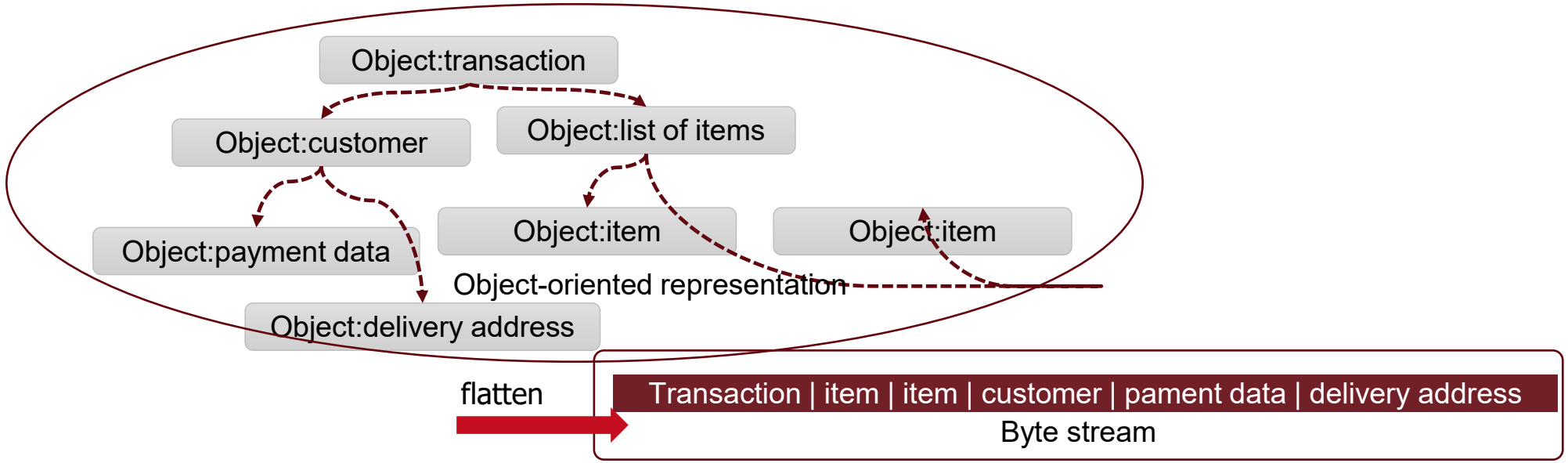
Failure Model (often described as a reliable transport service): 'more' reliable than UDP since it is stream-based, maintains knowledge about state of a connection

- Integrity: checksums to detect corrupted messages, sequence numbers to detect duplicates
- Validity: timeouts and retransmissions to deal with lost packets → it guarantees to deliver messages even if some packets are lost
- If #lost_packets exceeds a limit or network fails, TCP declares connection as broken → TCP not reliable as it not guarantees delivery in case of all failures
- Sending process cannot determine whether the network or the receiving process (failed before or after message processed!?) has failed

External Data Representation

Sockets only support transmission of byte-streams (TCP) or byte-arrays (UDP).

How to flatten complex data structures (e.g. interconnected objects) before sending and rebuilding after arrival?



External Data Representation

Sockets only support transmission of byte-streams (TCP) or byte-arrays (UDP)

How to handle heterogeneity – i.e. different representations of primitive values on different nodes? Or different character / video / audio encoding?

Address	0	1	2	3
Little Endian	D0	C0	B0	A0
Big Endian	A0	B0	C0	D0

Heterogeneity – Hardware
Little Endian vs. Big Endian given the value 0xA0B0C0D0

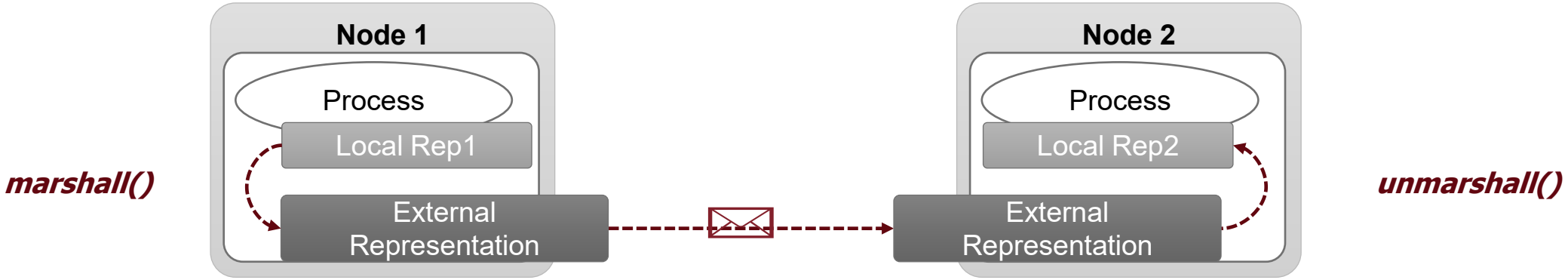
Address	0	1	2	3
C++	A	B	C	0
Pascal	3	A	B	C

Heterogeneity – Programming Language
C++ vs. Pascal given the string “ABC”

External Data Representation

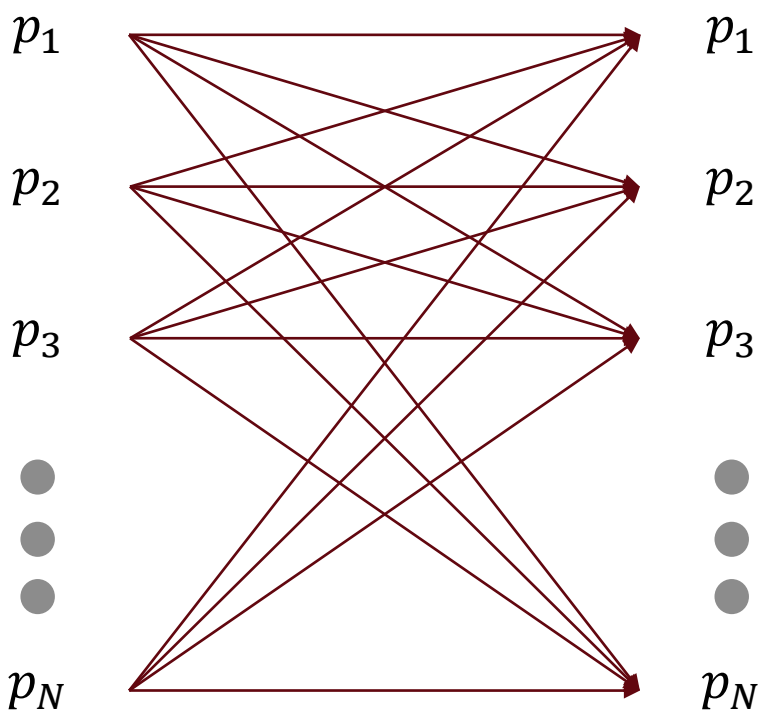
Data must be transformed (*local representation → external data representation*) as a standard for the representation of data structures and primitive values

- Marshalling: process of translating / transforming data structures and primitive values into the external data representation (includes flattening)
- Unmarshalling: inverse process of translating the external data representation to the local representation expected by the recipient



Two options for data transformation

1. Pairwise transformation between N local representations
   - Either sender or receiver transforms
   - One transformation per communication

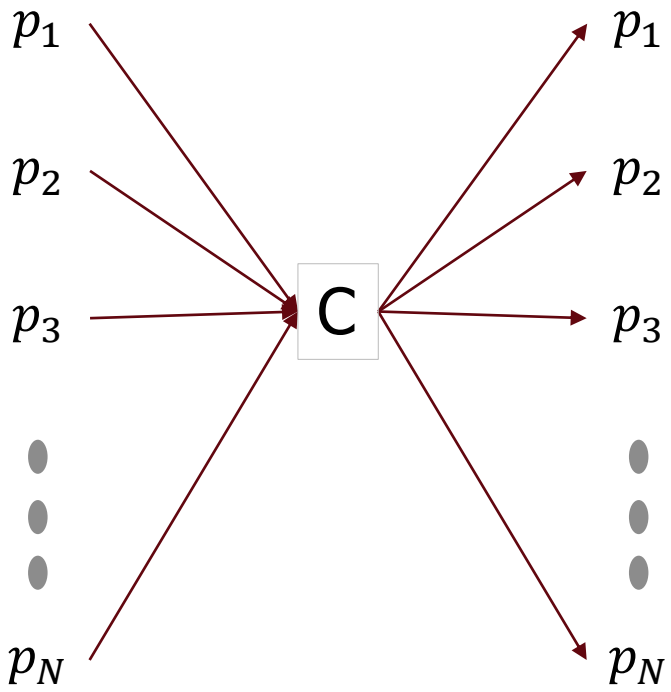


Transformations required given a distributed system with N heterogeneous nodes

Two options for data transformation

2. Canonical representation C
   - No knowledge about the representation of the remote peer required
   - Only 2xN transformations required (to C, from C)

   ⇒two transformations per communication required



Abstract Syntax Notation (ASN.1)

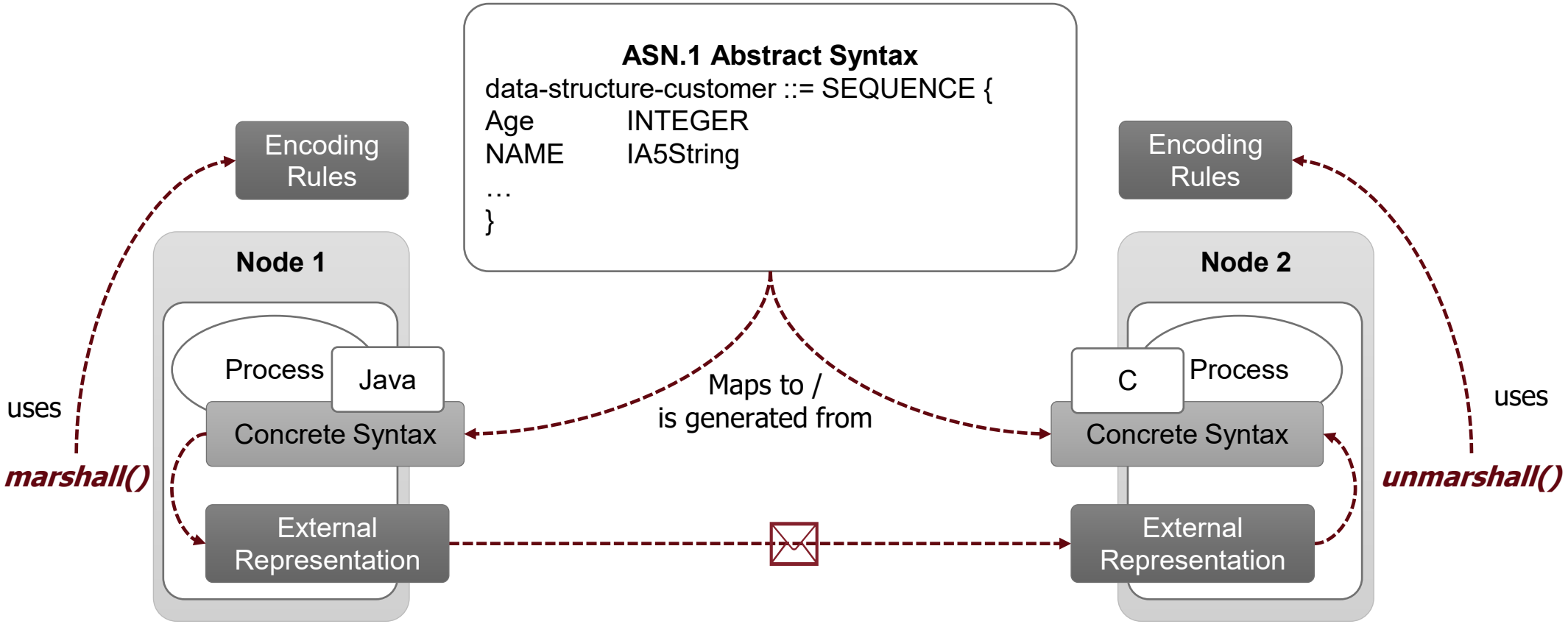
ISO standard for implementing External Data Representations

- Abstract Syntax: describes the common structure of data independent of the programming language
- Concrete Syntax: local representation dependent on the programming language
- Transfer Syntax: describes the representation of data during transmission
- Encoding Rules: describes the transformation from concrete to transfer syntax

Developers define abstract syntax

Compiler generates concrete syntax and routines required for encoding / decoding

Abstract Syntax Notation (ASN.1)



Examples for External Data Representation

Java Object Serialization

– Contains type information, Marshalling into binary format

Extensible Markup Language (XML)

– Contains type information, Marshalling into textual format, Human readable

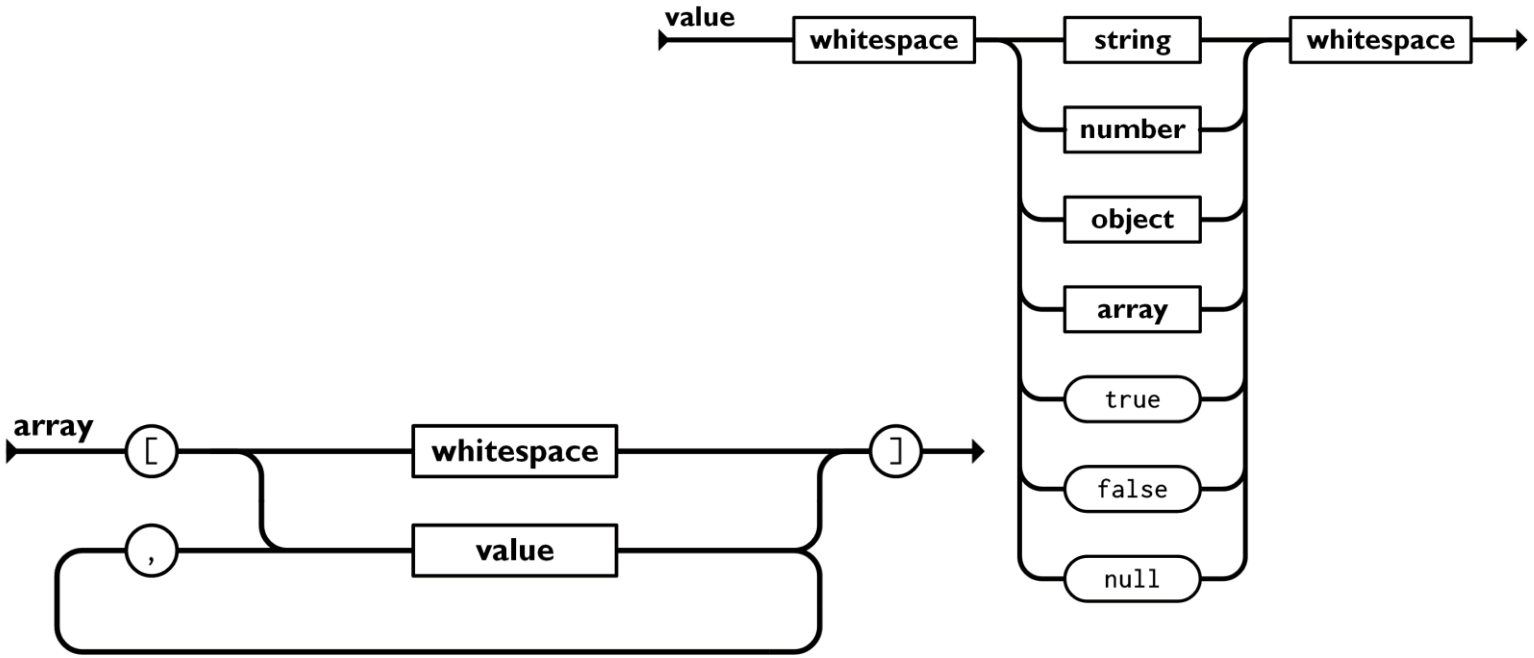
JavaScript Object Notation (JSON)

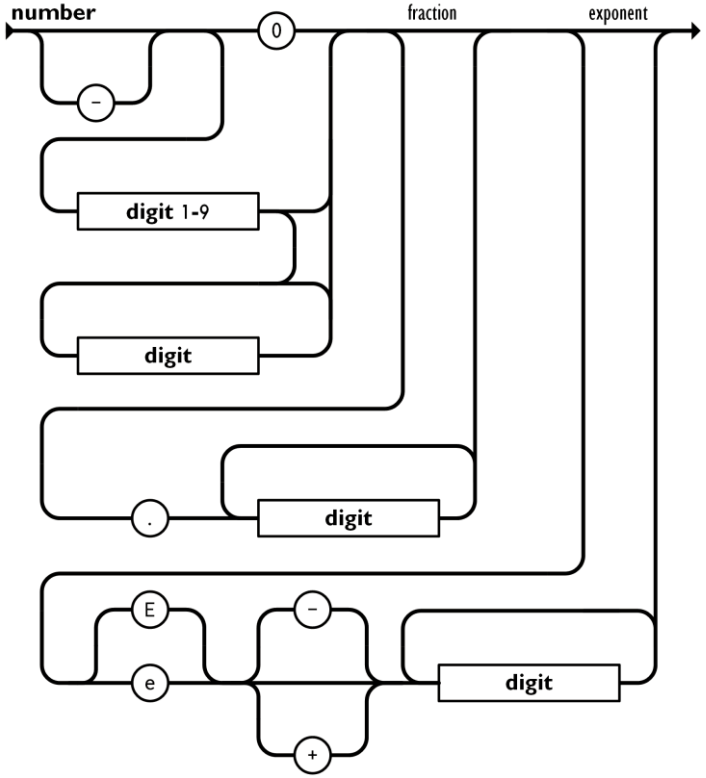
– Lightweight data-interchange format

Example JSON (JavaScript Object Notation = lightweight data-interchange format)

```
{"firstName": "John",
  "lastName": "Snow",
  "age": 25,
  "children": [],
  "spouse": null,
  "address": {
          "street": "7504 Taylor Drive",
          "city": "New York City",
          "state": "New York",
          "postalCode": "11238"
  },
phoneNumbers: [
    {
          "type": "mobile",
          "number": "212 555-3346"
    },
    {
          "type": "fax",
          "number": "646 555-4567"
    }
  ]
}
```

JSON Syntax (see json.org)

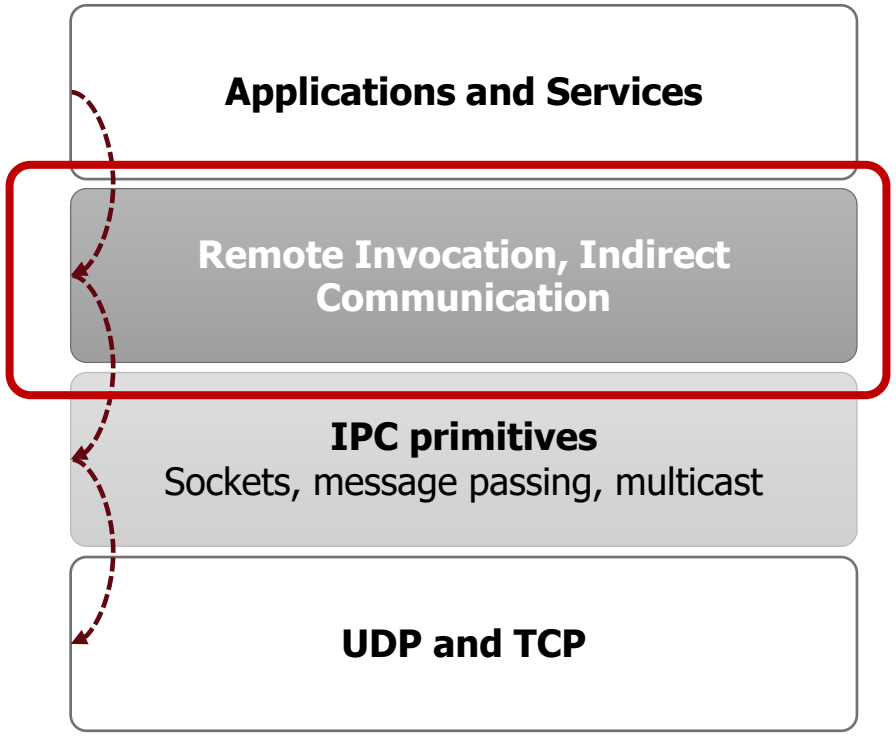




Message Queues

More sophisticated communication paradigms realized on top of IPC primitives

Message queues are a common example for indirect communication



Message Queues - Motivation

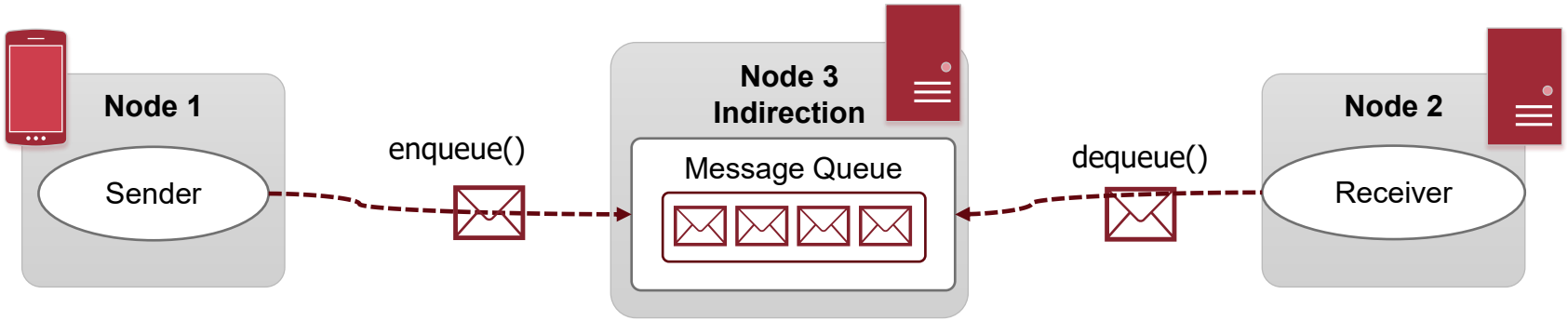
What happens if …

… a message is sent to a destination which is temporarily not available?

… a server fails while it processes a request?

… a message is lost in transit?

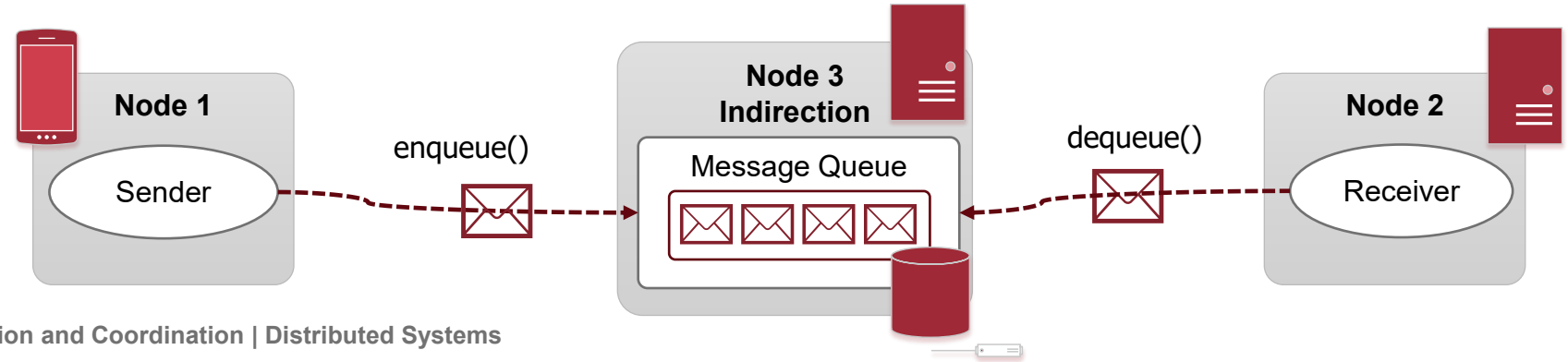
Solution: Introduce an indirection between sender and receiver that persistently buffers the messages!



Message Queues - Properties

Message queue = **named** destination as an **indirection** between sender / receiver

- Allows processes to execute and fail independently
- Masking process failures and communication failures
- Stores (queues) messages either volatile (unreliable in memory) or persistent (reliable – e.g. in a DB or state snapshotted to disk)
- Provides two basic operations similar to send/receive: enqueue/dequeue

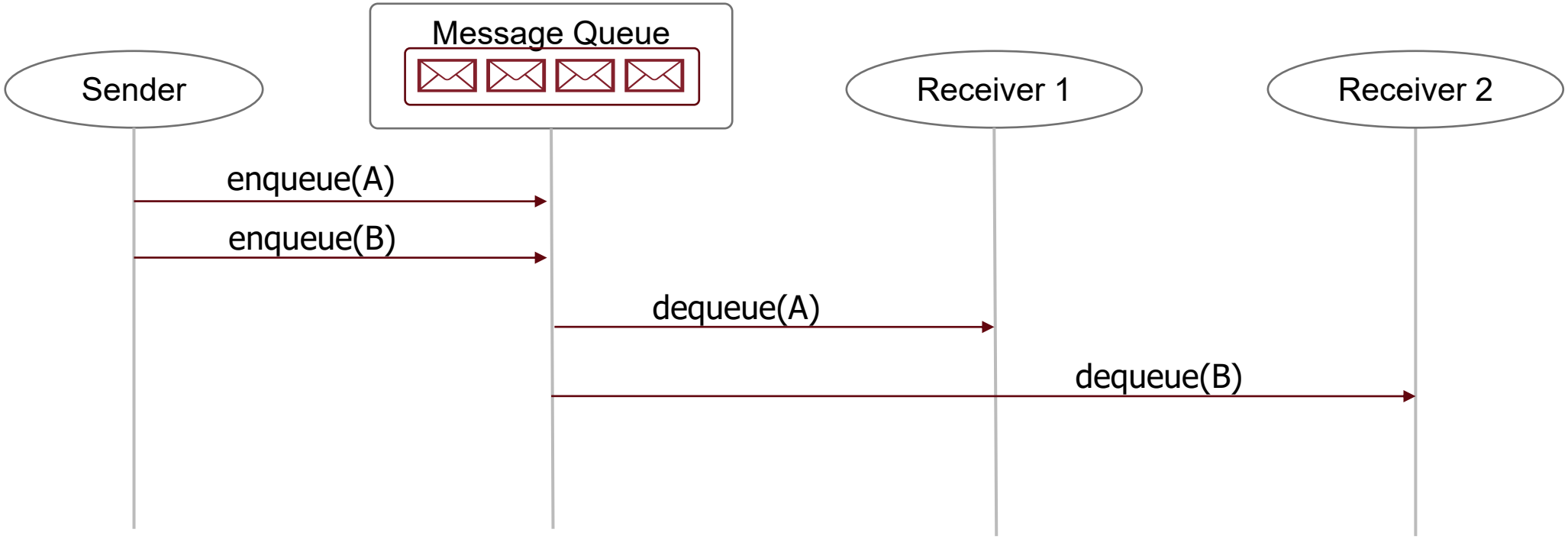


Message Queues - Properties

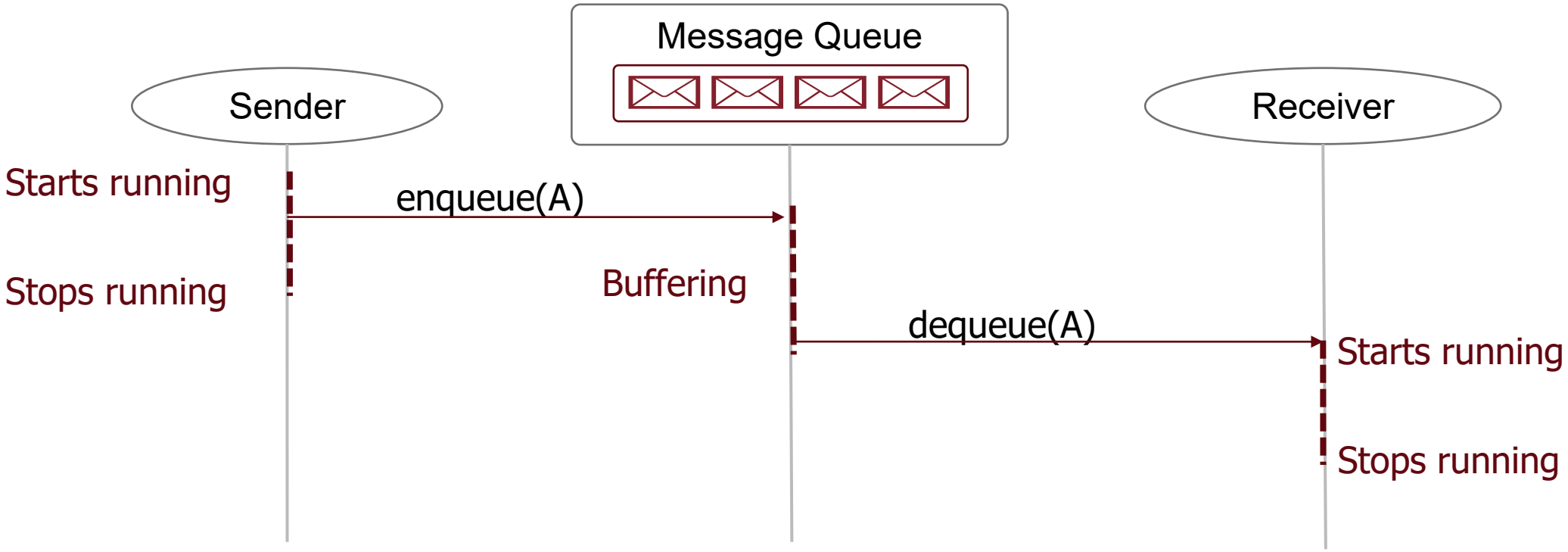
According to definition of indirect communication, message queues decouple sender and receiver in two dimensions

- Space decoupling: Sender does not need to know the actual receiver
- Time decoupling: Sender and receivers are not required to be available at same time
- Message Queues add flexibility at the expense of additional overhead

Message Queues - Space Decoupling



Message Queues - Time Decoupling



Message Queues – Messaging Patterns

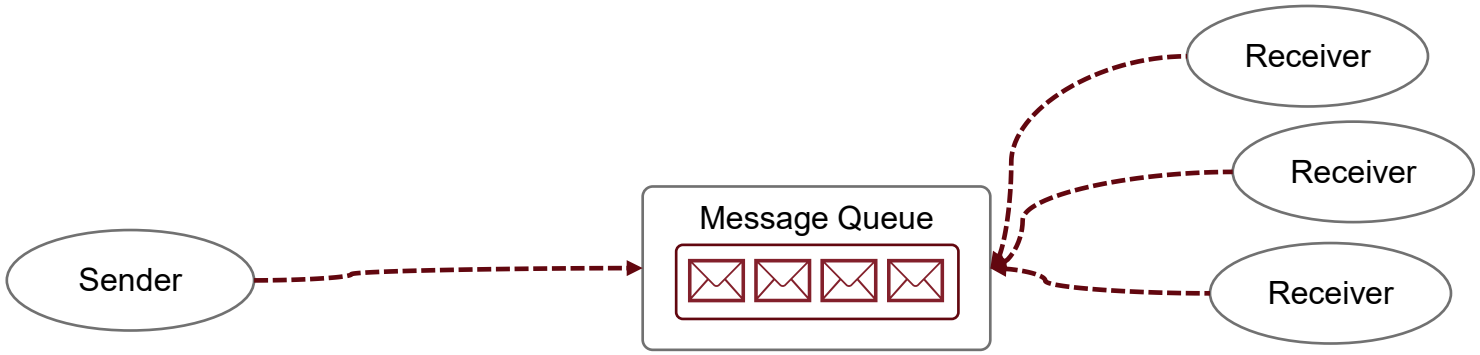
Besides decoupling of sender and receiver, queues offer four messaging patterns:

- One-to-One: One sender, one receiver

Message Queues – Messaging Patterns

Besides decoupling of sender and receiver, queues offer four messaging patterns:

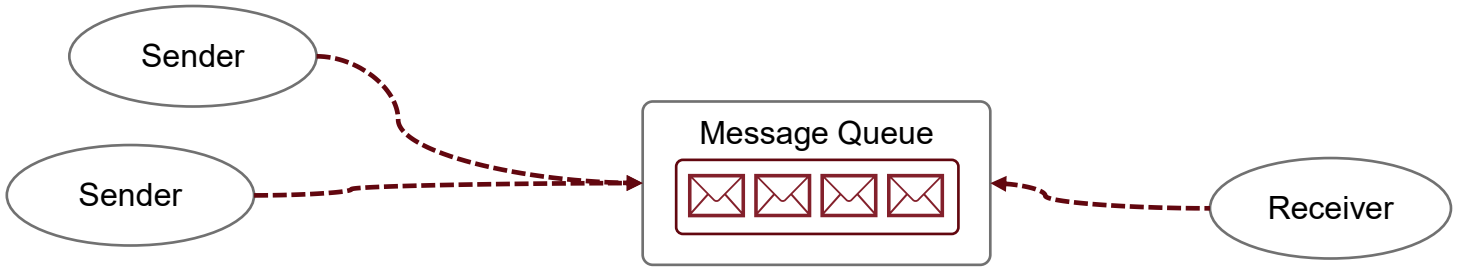
- One-to-One:
- One-to-Many: One sender, many receivers



Message Queues – Messaging Patterns

Besides decoupling of sender and receiver, queues offer four messaging patterns:

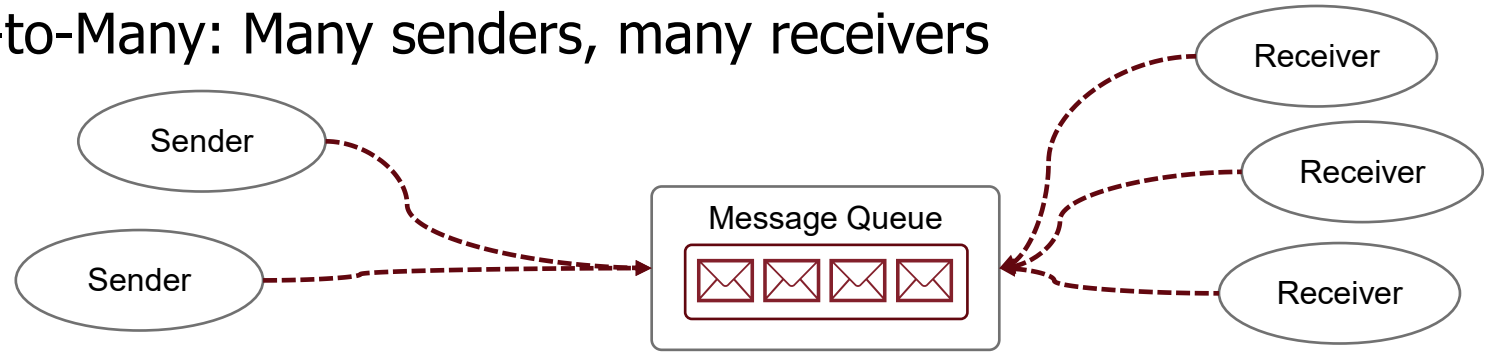
- One-to-One:
- One-to-Many:
- Many-to-One: Many senders, one receiver



Message Queues – Messaging Patterns

Besides decoupling of sender and receiver, queues offer four messaging patterns:

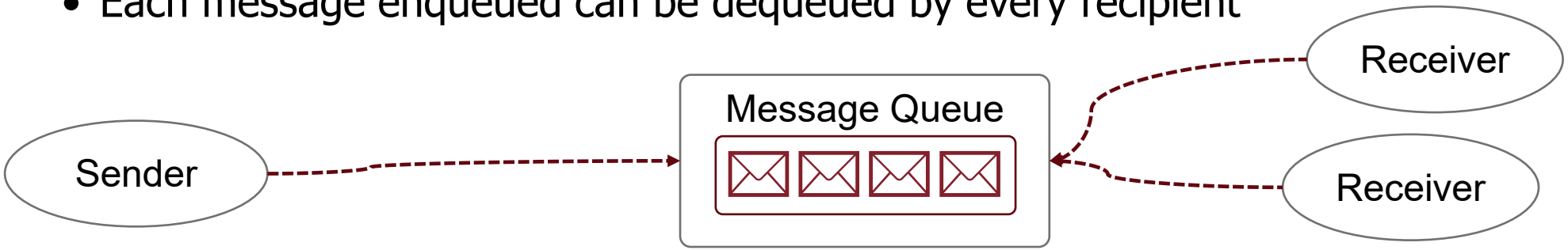
- One-to-One:
- One-to-Many:
- Many-to-One:
- Many-to-Many: Many senders, many receivers



Message Queues – Messaging Patterns

Multiple consumer queues make single messages available to multiple receivers/consumers determined through:

- Queue subscribers (implicit receivers): Consumers may "subscribe" to a queue
  - Each message enqueued can be dequeued by every subscriber
  - Can be combined with message selectors
- Message recipient list (explicit receivers): Determined by the sender / producer
  - Each message enqueued can be dequeued by every recipient



Message Queues – Message Properties

Messages in the queue can be accessed / de-queued using selection operations

- Options for message selection: first, next, or selectors for content-based retrieval
- Mode Remove: message is removed
- Mode Browse: message remains in queue and is available for other consumers

Common properties of queued messages

- Priority: message with higher priority is preferably forwarded
- Earliest de-queue time: message should not be de-queued before given time
- Expiration time: message is deleted from queue if it expires
- Correlation ID: used to match a request with the corresponding reply

Multicast

Pairwise exchange of messages is not optimal if communication takes place between a sending process and a group of receiving processes

- Multicast Operation sends a single message from one process to each member of a group of processes
- Membership is usually transparent to the sender

Multicast - Motivation

Multicast operations provide a useful infrastructure for several DS problems:

- Fault tolerance based on replicated services: Client requests are multicast to a group of replicated servers → clients are served even if some of the servers fail
- Service Discovery: Multicast messages can be used to locate available Discovery Services and look up particular services
- Better performance through replicated data: If replicated data changes, multicast messages can be used to propagate changes to all copies
- Event notifications: Notify a group of processes interested in certain events that a new event happened

Questions ?